

Term Project – Documentation

Study Room Reservation System: a database-backed reservation system

Dozent: Stephan Bökelmann sboekelmann@ep1.rub.de Technische Hochschule Georg Agricola,
Bochum

Author: Imane Tamouh

Module: Introduction to Database Management Systems · THGA Bochum

Stack: PostgreSQL · FastAPI · Docker Compose · tkinter (.deb)

Repository: <https://github.com/imanetamouh7-bit/study-room-reservation>

Inhaltsverzeichnis

Contents

1	Introduction and motivation	2
2	Data model	2
2.1	Relational schema	2
3	Database (PostgreSQL)	2
4	REST API (FastAPI)	3
5	Frontend (tkinter, packaged as .deb)	4
6	Operation and deployment	5
7	Reflection	6

1 Introduction and motivation

Students often need a quiet place to study, work on assignments or prepare for examinations. If study rooms are managed manually, it can be difficult to determine which rooms are available and whether a room has already been reserved for a particular period.

The Study Room Reservation System replaces manual reservation management with a relational database system. It manages students, study rooms, equipment and reservations, exposes the data through a REST API and provides a small desktop frontend.

Usage scenario. A student wants to reserve a study room for an examination preparation session. The student opens the frontend, enters the API URL and the X-API-Key and connects to the backend. The available study rooms are displayed together with their capacity. The student selects a room, enters the date and time and creates the reservation. If the selected room is already reserved for the requested time period, the system rejects the reservation and informs the student that another room or another time period has to be selected.

2 Data model

The domain has five entity types: students, study rooms, equipment, reservations and the associative entity connecting rooms with equipment.

A student can make many reservations over time, while each reservation belongs to one student. A room can be reserved many times over different time periods, while each reservation refers to one room.

A room can have many equipment types and an equipment type can occur in many rooms. This is an N:M relationship that is resolved by the `room_equipment` relation.

ER model (textual).

- *student* (1) – makes – (N) *reservation*
- *room* (1) – is reserved in – (N) *reservation*
- *room* (1) – is assigned – (N) *room_equipment*
- *equipment* (1) – occurs in – (N) *room_equipment*

Thus *room* N:M *equipment*, resolved through *room_equipment*.

2.1 Relational schema

Derived from the ER model, with primary keys underlined and foreign keys in italics:

- *student*(student_id, name, email)
- *room*(room_id, room_number, capacity)
- *equipment*(equipment_id, name)
- *reservation*(reservation_id, date, start_time, end_time, *student_id*, *room_id*)
- *room_equipment*(room_id, equipment_id)

Normalisation. Every non-key attribute depends only on the key of its table, and there are no transitive dependencies between non-key attributes.

Student information is stored only in *student*, room information only in *room*, and equipment information only in *equipment*. Reservations reference students and rooms by their keys instead of duplicating their data.

The N:M relationship between rooms and equipment is represented by the separate *room_equipment* relation.

The schema is therefore in 3NF.

3 Database (PostgreSQL)

The schema and seed data are created by SQL init scripts that PostgreSQL runs when the database is initially created.

The PostgreSQL service uses version 16 and stores its data in the Docker volume `pg_data`.

Listing 1: schema.sql (excerpt) – student, room and equipment

```

1 CREATE TABLE student (
2   student_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3   name VARCHAR(120) NOT NULL,
4   email VARCHAR(200) NOT NULL UNIQUE
5 );
6
7 CREATE TABLE room (
8   room_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
9   room_number VARCHAR(20) NOT NULL UNIQUE,
10  capacity INTEGER NOT NULL CHECK (capacity > 0)
11 );
12
13 CREATE TABLE equipment (
14   equipment_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
15   name VARCHAR(100) NOT NULL UNIQUE
16 );
17

```

Listing 2: schema.sql (excerpt) – reservation

```

1 CREATE TABLE reservation (
2   reservation_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3   date DATE NOT NULL,
4   start_time TIME NOT NULL,
5   end_time TIME NOT NULL,
6   student_id INTEGER NOT NULL,
7   room_id INTEGER NOT NULL,
8
9   CHECK (end_time > start_time),
10
11  FOREIGN KEY (student_id)
12  REFERENCES student (student_id)
13  ON DELETE RESTRICT
14  ON UPDATE CASCADE,
15
16  FOREIGN KEY (room_id)
17  REFERENCES room (room_id)
18  ON DELETE RESTRICT
19  ON UPDATE CASCADE
20 );
21

```

Listing 3: schema.sql (excerpt) – room equipment

```

1 CREATE TABLE room_equipment (
2   room_id INTEGER NOT NULL,
3   equipment_id INTEGER NOT NULL,
4
5   PRIMARY KEY (room_id, equipment_id),
6
7   FOREIGN KEY (room_id)
8   REFERENCES room (room_id)
9   ON DELETE CASCADE
10  ON UPDATE CASCADE,
11
12  FOREIGN KEY (equipment_id)
13  REFERENCES equipment (equipment_id)
14  ON DELETE CASCADE
15  ON UPDATE CASCADE
16 );
17

```

The `student` table stores four initial students. The `room` table contains four study rooms with capacities 4, 6, 8 and 12.

The equipment table contains Projector, Whiteboard, Monitor and Video Conference.

The seed data also creates room-equipment assignments and four initial reservations.

4 REST API (FastAPI)

The API is the only component that talks to the database. Read endpoints retrieve information from PostgreSQL, while the write endpoint requires the header `X-API-Key`.

Method	Path	Purpose	Key
GET	/	API status	–
GET	/rooms	list study rooms	–
GET	/reservations	reservations + JOIN	–
POST	/reservations	create reservation	yes
GET	/rooms/usage	room usage (JOIN + agg.)	–

The `/reservations` endpoint demonstrates a JOIN between reservation, student and room.

The `/rooms/usage` endpoint demonstrates a JOIN combined with an aggregation. It counts the reservations belonging to each room.

Listing 4: Query behind GET `/rooms/usage` (JOIN + aggregation)

```

1  SELECT
2      ro.room_id,
3      ro.room_number,
4      ro.capacity,
5      COUNT(r.reservation_id) AS reservation_count
6  FROM room AS ro
7  LEFT JOIN reservation AS r
8      ON ro.room_id = r.room_id
9  GROUP BY
10     ro.room_id,
11     ro.room_number,
12     ro.capacity
13  ORDER BY ro.room_id;
14

```

The write path is guarded by a dependency that checks the API key.

Listing 5: API-key guard

```

1  def verify_api_key(x_api_key: str = Header(...)):
2      if x_api_key != ENV["API_KEY"]:
3          raise HTTPException(
4              status_code=401,
5              detail="Invalid API key",
6              )
7

```

The reservation creation endpoint uses this dependency:

Listing 6: Protected POST `/reservations`

```

1  @app.post("/reservations", status_code=201)
2  def create_reservation(
3      reservation: ReservationCreate,
4      _: None = Depends(verify_api_key),
5  ):
6      ...
7

```

Before inserting a reservation, the backend checks whether another reservation for the same room and date overlaps with the requested time interval.

If an overlap exists, the API returns HTTP 409:

Listing 7: Reservation conflict

```

1  raise HTTPException(
2      status_code=409,
3      detail="Room is already reserved for this time period",
4      )
5

```

5 Frontend (tkinter, packaged as .deb)

The frontend is a small tkinter application with screens for available study rooms, new reservations, existing reservations and room usage.

It never touches the database directly – it only calls the API.

At startup a connection dialog asks for the API URL and the X-API-Key.

The current default API URL is:

```

1  http://127.0.0.1:8000
2

```

After a successful connection, the frontend displays the available study rooms.

The main screen provides four actions:

- New Reservation
- View Reservations
- Room Usage
- Disconnect

New Reservation

The reservation form contains:

- Date
- Start time
- End time
- Student ID
- Study room

The frontend sends the values to:

```
1 POST /reservations
2
```

A successful request displays the generated reservation ID.

If the backend returns HTTP 409, the frontend displays a warning that the room is already reserved for the selected time period.

View Reservations

The reservation view calls:

```
1 GET /reservations
2
```

The returned data is displayed in a table containing:

- reservation ID
- date
- start time
- end time
- student
- room

Room Usage

The room usage view calls:

```
1 GET /rooms/usage
2
```

The table displays the room ID, room number, capacity and number of reservations.

Packaging

The frontend source contains a PyInstaller specification file `main.spec`. The current project also contains generated frontend files in the `build` and `dist` directories.

For the final deployment, the frontend is packaged as a Debian package (`.deb`) so that it can be installed independently from the Docker Compose backend.

6 Operation and deployment

The backend runs as two containers, orchestrated by Docker Compose: a PostgreSQL database container and a FastAPI API container.

Credentials and configuration values are supplied through the `.env` file.

Listing 8: docker-compose.yml (excerpt)

```
1 services:
2
3 postgres:
4
5 image: postgres:16
6
7 restart: unless-stopped
8
9 environment:
10 POSTGRES_USER: ${POSTGRES_USER}
11 POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
12 POSTGRES_DB: ${POSTGRES_DB}
```

```
13
14     volumes:
15     - pg_data:/var/lib/postgresql/data
16     - ./database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
17     - ./database/seed.sql:/docker-entrypoint-initdb.d/02-seed.sql
18
19     healthcheck:
20     test:
21     [
22     "CMD-SHELL",
23     "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
24     ]
25     interval: 5s
26     timeout: 5s
27     retries: 5
28
29     api:
30
31     build: ./backend
32
33     restart: unless-stopped
34
35     env_file:
36     - .env
37
38     environment:
39     DB_HOST: postgres
40     DB_NAME: ${POSTGRES_DB}
41     DB_USER: ${POSTGRES_USER}
42     DB_PASSWORD: ${POSTGRES_PASSWORD}
43     API_KEY: ${API_KEY}
44
45     ports:
46     - "8000:8000"
47
48     depends_on:
49     postgres:
50     condition: service_healthy
51
52     volumes:
53     pg_data:
54
```

The PostgreSQL container uses the named volume `pg_data` to persist the database data.

The API container is built from the `backend` directory and is started only after the PostgreSQL health check succeeds.

Deployment steps

1. Start the database and API:

```
1     docker compose up -d
2
```

2. Check the running containers:

```
1     docker compose ps
2
```

3. Install the frontend Debian package:

```
1     sudo dpkg -i study-room-reservation_0.1.0_amd64.deb
2
```

4. Launch the frontend.

5. Enter the API URL and the X-API-Key in the connection dialog.

The FastAPI documentation can be accessed locally at:

```
1     http://127.0.0.1:8000/docs
2
```

7 Reflection

The separation into database, API and frontend provides a clear architecture for the Study Room Reservation System.

The database stores the application data and enforces important integrity constraints. The API provides the interface between the database and the graphical application.

The JOIN query used by the reservation endpoint demonstrates how related information can be retrieved without duplicating student and room data. The aggregation query used by the room usage endpoint demonstrates how reservation counts can be calculated directly in SQL.

The reservation conflict check was an important part of the application because two overlapping reservations must not be created for the same room.

Docker Compose simplifies the operation of the backend by starting PostgreSQL and FastAPI together. The health check ensures that the API waits for the database to become available.

The tkinter frontend provides the user with a simple interface for connecting to the API, viewing rooms and reservations, creating new reservations and viewing room usage.

Sources and reuse

Reused with attribution, as permitted:

- API-key guard and REST API structure – adapted from the DBMS course examples and DBMS project exercises.
- Docker Compose structure with PostgreSQL and FastAPI – based on the DBMS Docker and FastAPI lectures.
- tkinter connection dialog and Treeview-based frontend layout – based on the frontend concepts presented in the DBMS lectures.
- PyInstaller and Debian packaging approach – based on the course requirements for the frontend deployment.

The database schema, seed data and Study Room Reservation System application-specific functionality were developed for this project.